

## Assignment: Enhance Wrangler with Byte Size and Time Duration Units Parsers

### 1. Background

The CDAP Wrangler library currently parses various data types but lacks built-in support for easily handling units like Kilobytes (KB), Megabytes (MB), milliseconds (ms), or seconds (s). Users often need to perform calculations or conversions on columns representing data sizes or time intervals, which requires complex multi-step recipes.

This assignment aims to enhance the Wrangler core library by adding native support for parsing and utilizing byte size and time duration units within recipes. This involves modifying the grammar, updating the parsing logic, extending the API, and implementing a new directive to demonstrate the usage.

### 2. Objectives

- Integrate new lexer tokens (BYTE\_SIZE, TIME\_DURATION) into the Wrangler grammar.
- Update the relevant Java code (wrangler-api, wrangler-core) to handle these new token types.
- Implement a new aggregate directive (aggregate-stats) that utilizes these new types.
- Develop comprehensive test cases, including aggregation scenarios using the new units.

### 3. Detailed Tasks

- **Fork the repo** - <https://github.com/data-integrations/wrangler> to your own github handle. **All changes should be committed back to this repository with a section in Readme.md about the usage of these 2 new parsers.**
- **(a) Grammar Modification (wrangler-core/src/main/antlr4/.../Directives.g4):**
  - Add the lexer rules for BYTE\_SIZE and TIME\_DURATION (including helper fragments BYTE\_UNIT, TIME\_UNIT).
  - Modify relevant *parser* rules (e.g., value, or create new specific rules like `byteSizeArg`, `timeDurationArg`) to accept BYTE\_SIZE and TIME\_DURATION tokens where appropriate for directive arguments.
  - Regenerate the ANTLR Java parser/lexer code using the appropriate build process (e.g., `mvn compile`).
- **(b) API Updates (wrangler-api module):**
  - Create new Java classes `ByteSize.java` and `TimeDuration.java` extending `Token` [`wrangler-api/src/main/java/io/cdap/wrangler/api/parser/Token.java`]

- These classes should parse the token string (e.g., "10KB", "150ms") in their constructor.
  - Provide methods to retrieve the value in a canonical unit (e.g., long getBytes() for ByteSize)
- Add BYTE\_SIZE and TIME\_DURATION to the Token Types
- Update usage definition and token definition to support specifying these new token types as valid directive arguments.
- **(c) Core Parser Updates (wrangler-core module):**
  - Modify by adding visit methods for the parser rules created/modified in step 3a (e.g., visitByteSizeArg, visitTimeDurationArg, or modifying visitValue if applicable).
  - Hint : chk ctx.getText().
  - Add the created token instances to the TokenGroup.
- **(d) New Directive Implementation (wrangler-core module):**
  - Create a new directive class, which can do aggregation, implementing the Directive interface
  - **UsageDefinition (define()):** The directive should accept at least four arguments:
    1. ColumnName (source column with byte sizes).
    2. ColumnName (source column with time durations).
    3. ColumnName (target column name for total size).
    4. ColumnName (target column name for total or average time).
 Optionally, add arguments to specify output units (e.g., 'MB', 'GB', 'seconds', 'minutes') or aggregation type (total, average).
  - **Initialization :** Store the source and target column names provided in the arguments.
  - **Execution :**
    - This directive should operate as an *aggregate*. So you need a store to accumulate totals. Think about it. Hint : Chk ExecutorContext
    - For each row:
      - Read the byte size value from the source size column.
      - Read the time duration value from the source time column.
      - Add these values (converted to canonical units like bytes and nanoseconds) to running totals stored in the Store.
    - **Finalization :** This method (if using RecipePipeline's aggregation capabilities) or logic within the last execute call needs to:
      - Retrieve the final totals from the Store.
      - Perform unit conversions if required by arguments (e.g., convert total bytes to MB, total nanoseconds to seconds).

- Return a *single* new Row containing the target columns with the calculated aggregate values (e.g., total\_size\_mb, total\_time\_sec).
- **(e) Testing (wrangler-core module):**
  - Add unit tests for the ByteSize and TimeDuration classes, verifying correct parsing and canonical value retrieval for various inputs (e.g., "10kb", "1.5MB", "5ms", "2.1s").
  - Add parser tests (e.g., in GrammarBasedParserTest.java or RecipeCompilerTest.java to ensure recipes using the new syntax are parsed correctly and invalid syntax is rejected.
  - Add comprehensive unit tests for the new AggregateStats directive. See the Test Case Specification below.

#### 4. Test Case Specification: Aggregation

- **Input Data:** Create a list of Row objects or parse a simple file representing sample log or transaction data.  
E.g You can use TestingRig
- **Recipe:**

```
String[] recipe = new String[] {
    // Example: Aggregate size (output MB), total time (output seconds)
    "aggregate-stats :data_transfer_size :response_time total_size_mb
total_time_sec"
    // Add variations for average, median, p95, p99 and different output units if
    implemented
};
```
- **Execution:** Use TestingRig.execute(recipe, rows) to run the recipe.
- **Expected Output:** Assert that the output contains a *single* row with the correctly calculated aggregate values.
  - **Size Calculation:** Sum all data\_transfer\_size values (converted to bytes) and then convert the final sum to Megabytes (MB) for the total\_size\_mb column (using 1 MB = 1024 \* 1024 bytes or 1000 \* 1000 bytes - be consistent!).
  - **Time Calculation:** Sum all response\_time values (converted to nanoseconds or milliseconds) and then convert the final sum to seconds for the total\_time\_sec column. (If implementing average, divide by the number of rows before unit conversion).
  - Example assertion structure:
 

```
// results = TestingRig.execute(recipe, rows);
Assert.assertEquals(1, results.size());
Assert.assertEquals(expectedTotalSizeInMB,
```

```
results.get(0).getValue("total_size_mb"), 0.001); // Use tolerance for
float/double
Assert.assertEquals(expectedTotalTimeInSeconds,
results.get(0).getValue("total_time_sec"), 0.001);
```

## 5. Deliverables

- Assignment will be only evaluated if committed to github.
- Modified Directives.g4 file.
- All new and modified Java source files (.java) within wrangler-api and wrangler-core modules.
- All new and modified unit test files (.java) within wrangler-core.
- Evidence of successful build and test execution.

## 6. Evaluation Criteria (Example)

- Correctness: Do the new tokens parse correctly? Does the directive compute aggregates accurately? Do tests pass?
- Code Quality: Is the code well-formatted, commented, and easy to understand? Are existing patterns followed?
- Robustness: Does the code handle edge cases (e.g., zero values, large numbers, different unit cases)?
- Test Coverage: Are the new features adequately tested?